

西安交通大学

课程实验报告

实验名称: 区块链

姓名: 林圣翔

学号: 2223312202

专业班级: 计试 2201

指导教师: 马小博教授

报告日期: 2025.12.20



一、实验目的

1. 了解区块链。
2. 搭建一个自己的区块链。

二、实验平台

1. 宿主操作系统：Windows 10
2. Linux 环境：Ubuntu 20.04 LTS（基于 WSL 2）

三、实验原理

区块链是一种去中心化的分布式账本技术。它通过在多个参与节点间建立共识，共同维护一个按时间顺序链接、公开透明且难以篡改的数据链。其核心特征包括去中心化、链式存储结构、不可篡改性和安全可信性，融合了分布式存储、点对点传输、共识机制与密码学等多种技术。

区块链系统由三个基本要素构成：交易、区块和链。“交易”是引发账本状态变化的最小操作单位；“区块”则打包记录一段时间内已验证的交易，代表网络共识；“链”则是将这些区块按序通过密码学哈希值串联起来的历史日志。每个新区块都包含前序区块的哈希值，形成紧密的链式结构。任何对历史数据的篡改都会导致哈希值变化，并被后续区块验证所发现。若要成功篡改，必须同时修改该区块之后的所有区块并重新获得全网共识，这在计算和实践上几乎不可能，从而确保了数据的不可篡改性。

共识机制是区块链网络协调各分布式节点、就账本状态达成全局一致的核心协议。它通过一套预设的规则（如工作量证明、权益证明等）来解决在无中心协调者情况下的数据同步与信任问题，确保所有节点维护的账本副本最终一致。密码学技术，特别是非对称加密与哈希函数，为系统提供了至关重要的安全保障：非对称加密实现了交易的身份认证与数据机密性，而哈希函数则构建了区块之间的不可篡改的链式结构。智能合约作为部署在区块链上的可编程代码，其自动执行与不可篡改的特性，将区块链的功能从静态的价值记录账本，扩展为一个能够处理复杂逻辑、实现条件化价值转移的“去信任”分布式计算平台。这三者——共识机制、密码学与智能合约——共同构成了支撑区块链作为可信基础设施的核心技术支柱。

比特币作为区块链的首个成功应用，其发展历程（从创世区块、关键版本迭代到应对硬分叉挑战）完整展现了该技术从概念验证到成熟落地的关键阶段，为整个生态奠定了基础。如今，区块链技术已超越数字货币范畴，广泛应用于金融科技、供应链管理、数字身份认证等领域，并随着跨链、隐私计算等技术的发展，持续演进为构建未来数字化社会信任体系的重要基础设施。

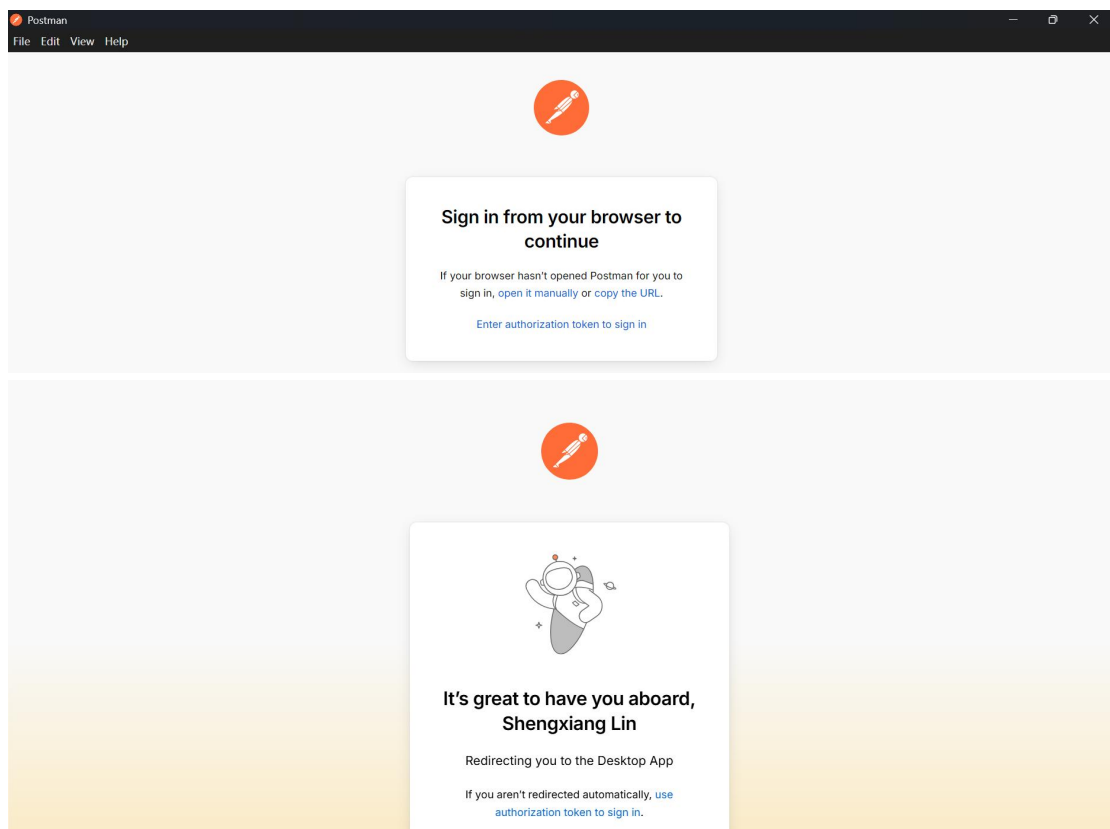
四、实验步骤

1. 创建自己的区块链

1.1. 查看当前 Ubuntu 系统中 Python、Flask 及 Requests 库的版本信息。

```
linshengxiang@sxLin:~$ python3 --version
Python 3.8.10
linshengxiang@sxLin:~$ pip show flask
Name: flask
Version: 3.0.3
Summary: A simple framework for building complex web applications.
Home-page: None
Author: None
Author-email: None
License: None
Location: /home/linshengxiang/.local/lib/python3.8/site-packages
Requires: importlib-metadata, itsdangerous, Jinja2, blinker, Werkzeug, click
Required-by: flask-sqlalchemy, Flask-RESTful, Flask-Migrate, Flask-JWT-Extended, Flask-Cors
linshengxiang@sxLin:~$ pip show requests
Name: requests
Version: 2.22.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache 2.0
Location: /usr/lib/python3/dist-packages
Requires:
Required-by: sphinx, scholarly, free-proxy, azure-core
```

1.2. 在 Windows 桌面环境打开 Postman。



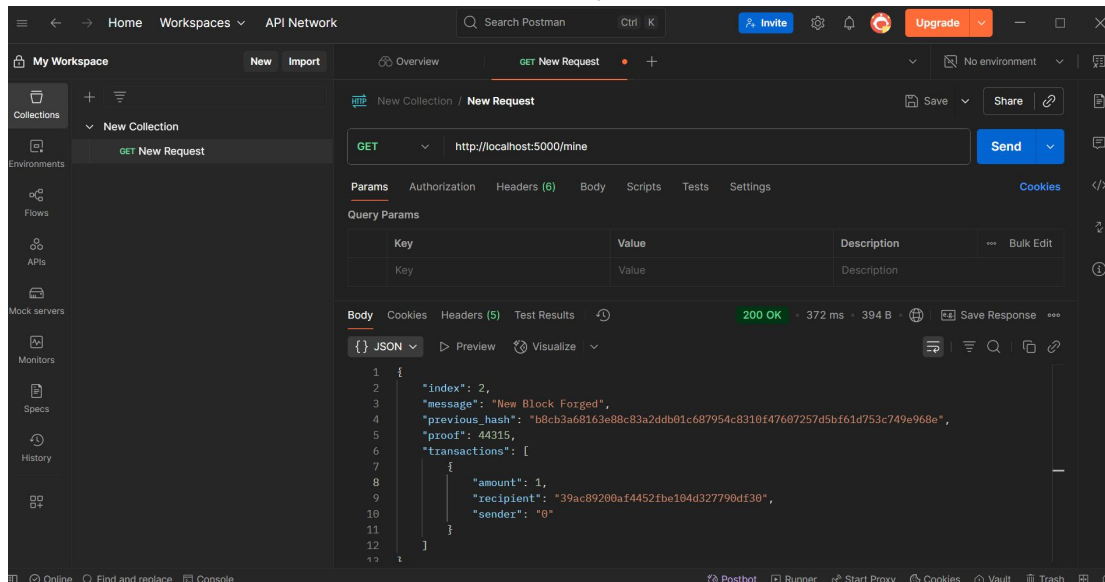
1.3. 创建 blockchain.py，参考 <https://segmentfault.com/a/1190000014483104> 完成代码编写，运行 python3 blockchain.py 命令。

```

linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (3.0.4) do
esn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported "
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit

```

1.4.在 Postman 中输入新建一个请求，并访问 <http://localhost:5000/mine>。

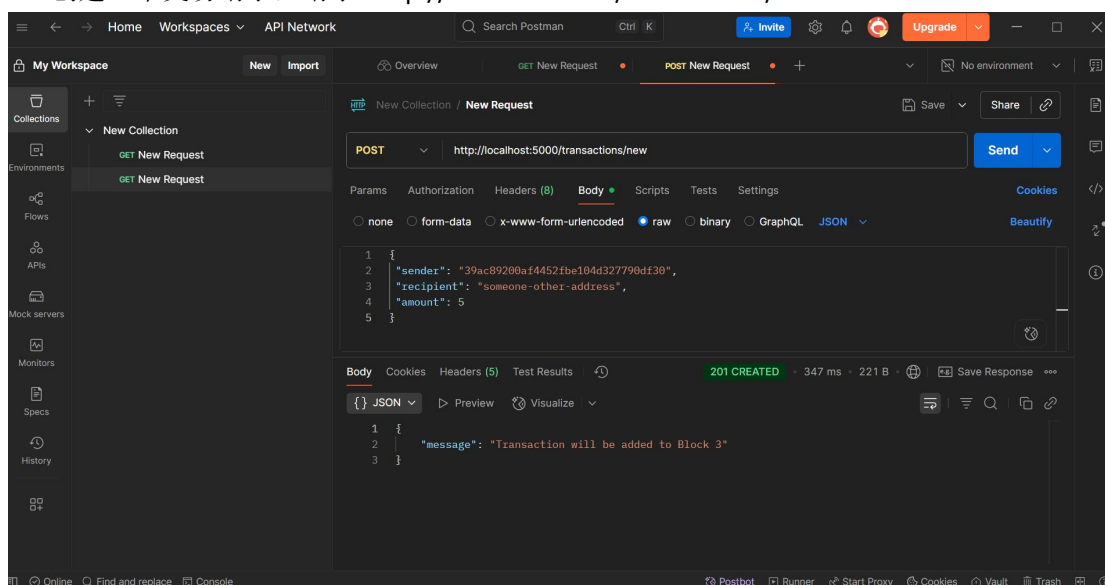


```

linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (3.0.4) do
esn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported "
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit
127.0.0.1 - - [19/Dec/2025 16:19:20] "GET /mine HTTP/1.1" 200 -

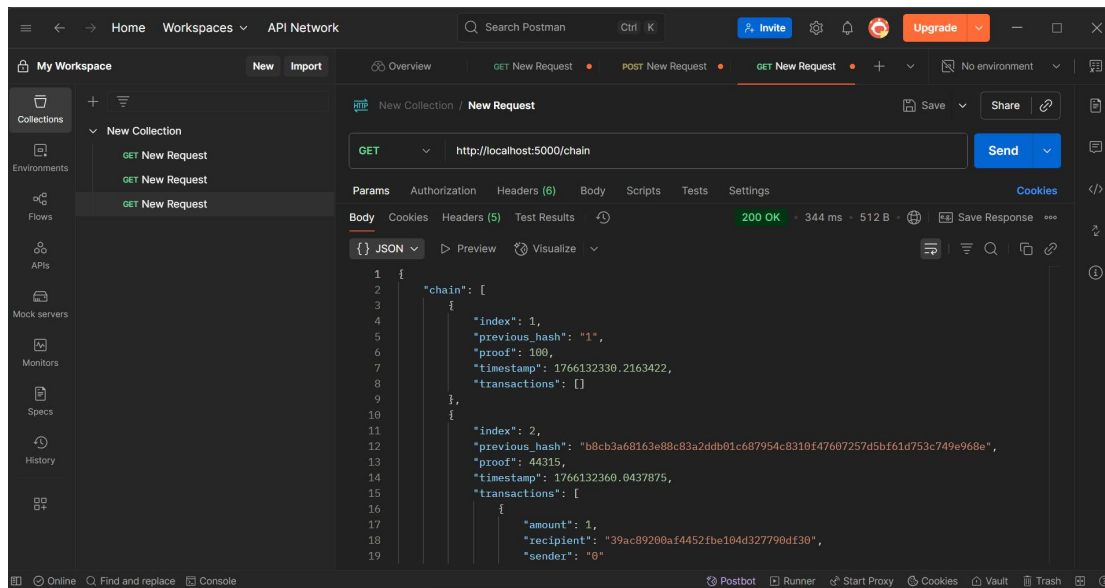
```

1.5.创建一个交易请求，请求 <http://localhost:5000/transactions/new>（POST）。



```
linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (3.0.4) do
esn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported "
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit
127.0.0.1 - - [19/Dec/2025 16:19:20] "GET /mine HTTP/1.1" 200 -
127.0.0.1 - - [19/Dec/2025 16:21:53] "POST /transactions/new HTTP/1.1" 201 -
```

1.6.通过请求 `http://localhost:5000/chain` 可以得到当前所有的块信息。



```
linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or cha
rdet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported "
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serv
er instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit
127.0.0.1 - - [19/Dec/2025 16:19:20] "GET /mine HTTP/1.1" 200 -
127.0.0.1 - - [19/Dec/2025 16:21:53] "POST /transactions/new HTTP/1.1" 201 -
127.0.0.1 - - [19/Dec/2025 16:25:20] "GET /chain HTTP/1.1" 200 -
```

2.节点一致性

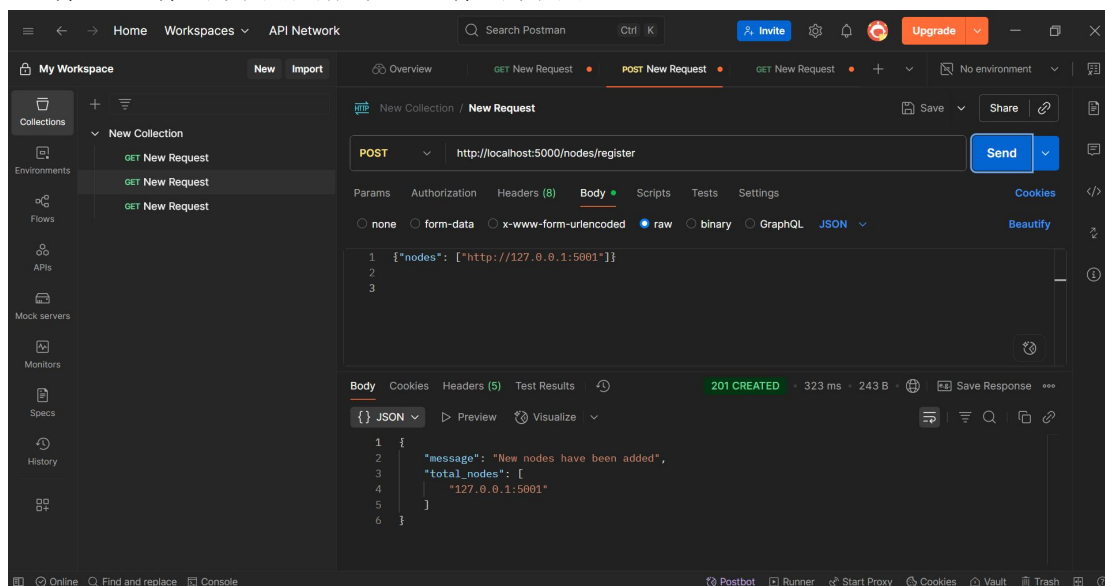
2.1.打开两个终端，分别执行 `python3 blockchain.py`（默认端口 5000）和 `python3 blockchain.py -p 5001`（指定端口 5001）。

```
linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or cha
rdet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported "
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serv
er instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit
```



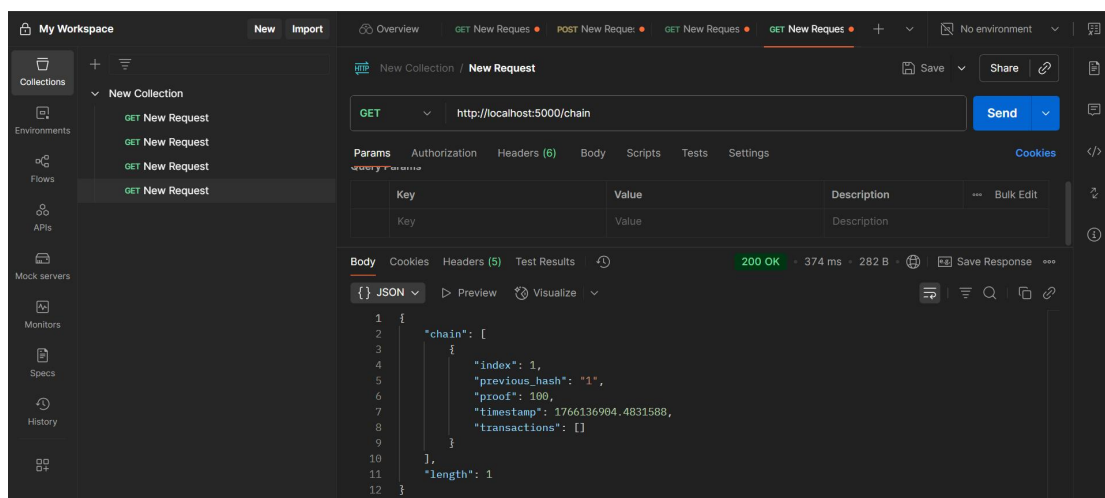
```
linshengxiang@sxLin:~$ python3 blockchain.py -p 5001
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported version".format(urllib3.__version__, chardet.__version__), RequestsDependencyWarning)
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.3.30:5001
Press CTRL+C to quit
```

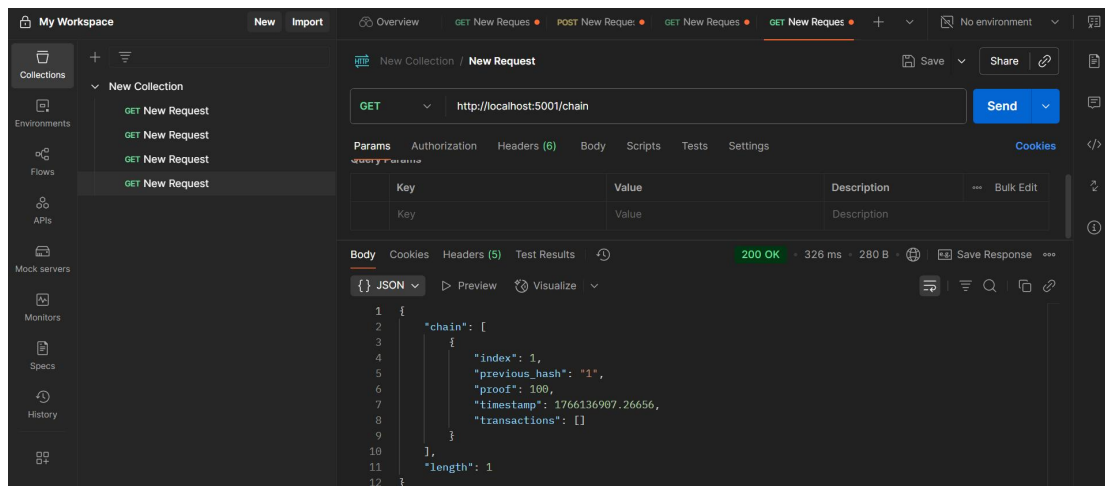
2.2.将 5001 端口的节点注册到 5000 端口的节点上。



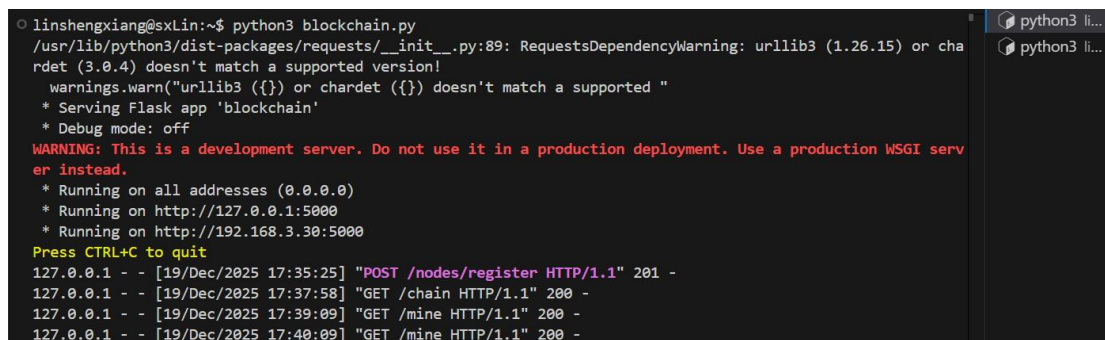
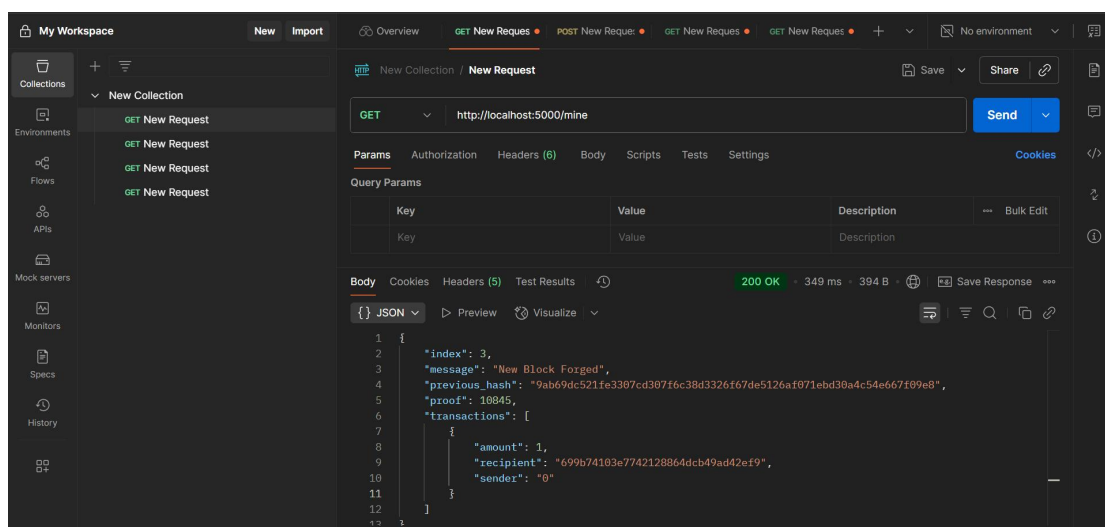
```
linshengxiang@sxLin:~$ python3 blockchain.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (1.26.15) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), doesn't match a supported version".format(urllib3.__version__, chardet.__version__), RequestsDependencyWarning)
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.3.30:5000
Press CTRL+C to quit
127.0.0.1 - - [19/Dec/2025 17:35:25] "POST /nodes/register HTTP/1.1" 201 -
```

2.3.通过查询，可以观察到端口 5000 和 5001 对应的两条区块链。在当前阶段，每条链的长度都仅为 1，这说明网络刚刚启动，仅存在唯一的创世块。

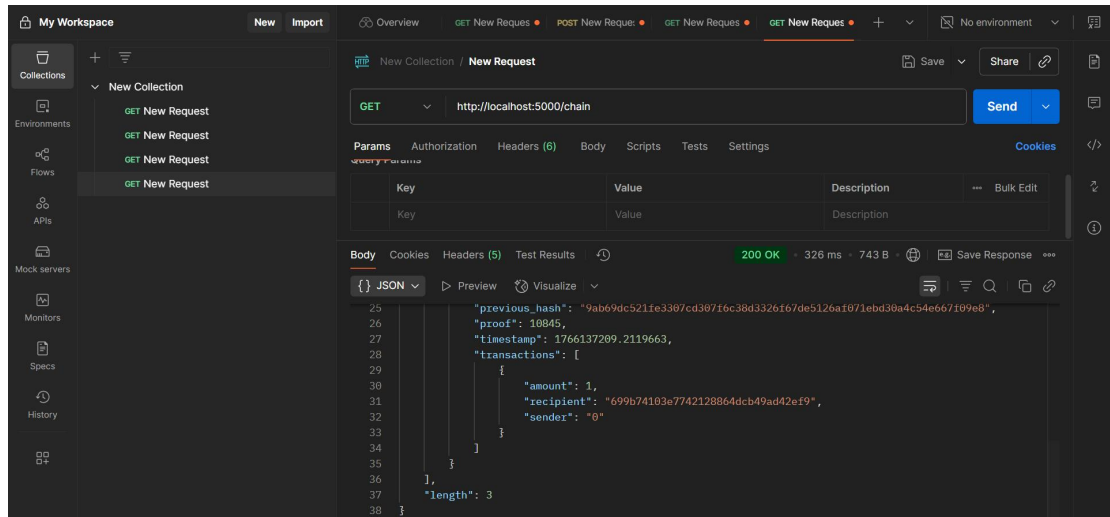




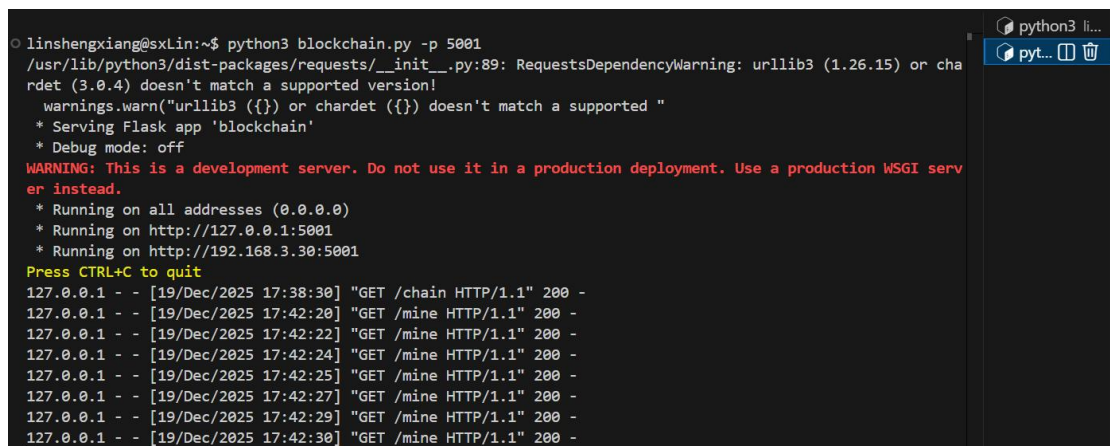
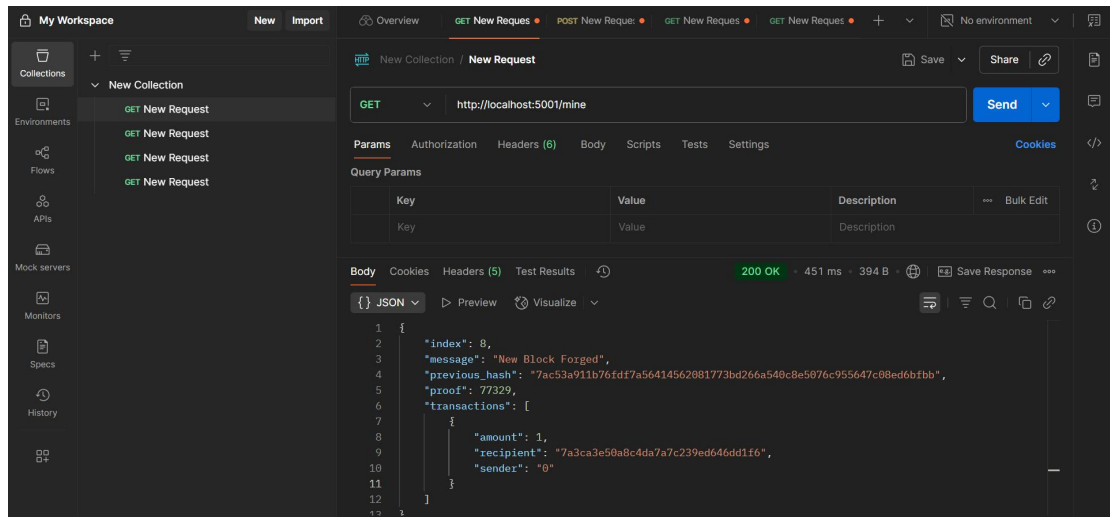
2.4.在端口 5000 的节点上连续进行 2 次挖矿操作。



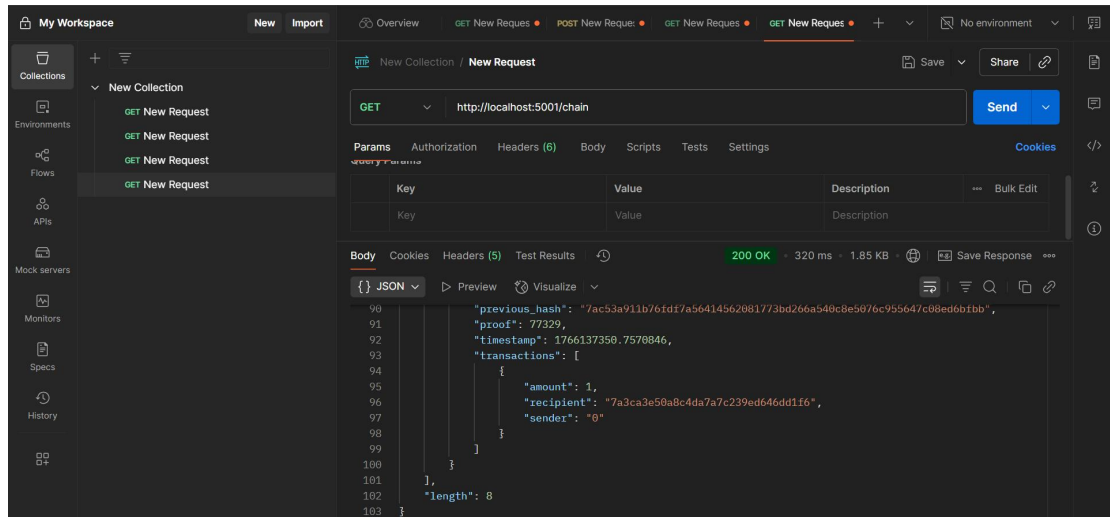
再次查询该链，会发现其长度已从最初的 1 增加到了 3。



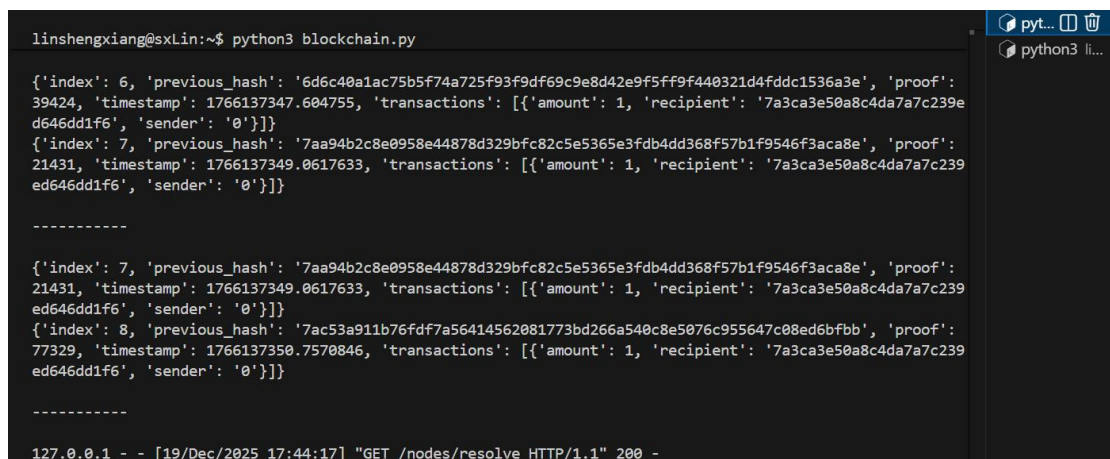
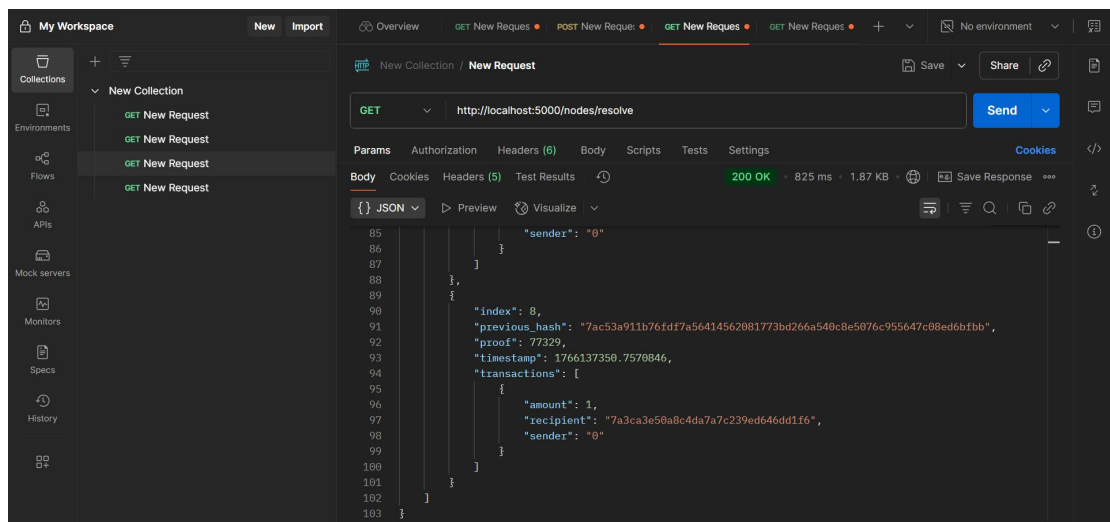
2.5.在端口 5001 的节点上连续进行 7 次挖矿操作。



再次查询该链，会发现其长度已从最初的 1 增加到了 8。



2.6.调用端口 5000 节点的 `resolve` 接口后，该节点成功更新至一条更长的区块链（共 8 个区块），这标志着跨节点的区块链一致性已成功实现。



五、思考题

1.写出一个 block 中 index、timestamp（时间）、transactions（交易）、proof（工作量证明）、

`previous_hash` 各自的用处。

①`index`: 数据类型为整数, 表示区块序号, 即该区块在链中的位置(高度)。创世区块的 `index` 为 1, 后续区块依次递增。它用于快速定位和验证区块顺序。在 `new_block` 方法中, 其值为 `len(self.chain) + 1`。

②`timestamp`: 浮点数, 表示区块创建时间, 即记录区块被创建(或挖出)的时间点(Unix 时间戳)。它确保了区块在时间维度上的先后顺序, 是区块链历史记录不可篡改的基础之一。在 `new_block` 中通过 `time()` 自动生成。

③`transactions`: 数据类型为列表, 表示交易数据容器, 即存储在该区块内打包的所有交易记录。它是区块链作为分布式账本的核心数据载体。每当新区块生成时, `self.current_transactions` 列表中的待处理交易会被打包进此字段, 随后该临时列表被清空。

④`proof`: 数据类型为整数, 表示工作量证明, 即矿工通过“挖矿”计算到的一个符合网络难度目标的随机数。它是共识算法(如 PoW)的结果, 证明了为生成此区块所投入的计算资源, 是区块链实现去中心化信任和防止篡改的关键。其数值通过 `proof_of_work` 方法计算得出。

⑤`previous_hash`: 数据类型为字符串, 表示前驱区块哈希, 即前一个区块所有数据经过哈希算法(如 SHA-256)计算后得到的唯一“数字指纹”。此字段将区块按顺序加密链接, 任何对历史区块的篡改都会导致其哈希值变化, 从而使后续所有区块失效, 以此保障整条链的完整性与不可篡改性。在 `new_block` 方法中, 其值来自参数或通过 `hash(self.chain[-1])` 自动计算。

2. 这篇文章中的 `proof_of_work` 写的有一点问题, 请说明它有什么问题, 可能导致什么结构, 并进行修正, 附在实验报告中。

```
def proof_of_work(self, last_proof):
    """
    Simple Proof of Work Algorithm:
    - Find a number p' such that hash(pp') contains leading 4 zeroes, where p is the previous p'
    - p is the previous proof, and p' is the new proof
    :param last_proof: <int>
    :return: <int>
    """
    proof = 0
    while self.valid_proof(last_proof, proof) is False:
        proof += 1
    return proof

    @staticmethod
    def valid_proof(last_proof, proof):
        """
        Validates the Proof: Does hash(last_proof, proof) contain 4 leading zeroes?
        :param last_proof: <int> Previous Proof
        :param proof: <int> Current Proof
        :return: <bool> True if correct, False if not.
        """
        guess = f'{last_proof}{proof}'.encode()
        guess_hash = hashlib.sha256(guess).hexdigest()
```

```
return guess_hash[:4] == "0000"
```

该实现只使用了前一个区块的 **last_proof** 和当前尝试的 **proof** 来计算哈希，但没有包含前一个区块的哈希值。这使得工作量证明的计算与前一个区块的内容（即区块链的历史）没有紧密绑定，可能导致风险如下。

①计算哈希时缺少上下文关联性：如果只基于两个证明值计算哈希，那么同样的工作量证明结果可能在不同的区块链历史中重复出现。攻击者可能利用这个弱点，在不同分支上重用相同的工作量证明

②无法确保链的连续性：没有将前一个区块的哈希纳入计算，意味着当前区块的工作量证明不依赖于前一个区块的全部内容，这降低了链的防篡改性，因为篡改前一个区块不会影响当前区块的工作量证明难度。

③不符合区块链设计原则：区块链的核心是每个区块都依赖于前一个区块，形成一个不可分割的链条。这个实现破坏了这种依赖关系，使区块之间联系不够紧密。

修正思路：①参数类型改变，**proof_of_work** 方法现在接收整个 **last_block** 字典作为参数，而不是仅接收 **last_proof** 整数，这样可以同时获取前一个区块的证明值（**last_block['proof']**）和前一个区块的哈希值（**self.hash(last_block)**）；②验证逻辑增强，**valid_proof** 方法增加了一个参数 **last_hash**，哈希计算时组合了 **last_proof**、**proof** 和 **last_hash** 三个要素。修正后的代码如下。

```
def proof_of_work(self, last_block):
    """
    改进的工作量证明算法：
    - 找到一个数字 p'，使得 hash(p, p', last_hash) 的前 4 位为 0
    - 其中 p 是前一个区块的工作量证明，p' 是新的证明，last_hash 是前一个区块的哈希值
    :param last_block: <dict> 前一个区块
    :return: <int> 新的工作量证明
    """
    # 获取前一个区块的证明和哈希
    last_proof = last_block['proof']
    last_hash = self.hash(last_block)
    proof = 0
    while self.valid_proof(last_proof, proof, last_hash) is False:
        proof += 1
    return proof

@staticmethod
def valid_proof(last_proof, proof, last_hash):
    """
    验证工作量证明：hash(last_proof, proof, last_hash) 是否以 4 个 0 开头？
    :param last_proof: <int> 前一个工作量证明
    :param proof: <int> 当前工作量证明
    :param last_hash: <str> 前一个区块的哈希值
    :return: <bool> 验证通过返回 True，否则返回 False
    """
    # 将三个要素组合：前一个证明、当前证明、前一个区块哈希
```

```
guess = f'{last_proof}{proof}{last_hash}'.encode()
guess_hash = hashlib.sha256(guess).hexdigest()
# 检查哈希值是否以 4 个 0 开头（可根据需要调整难度）
return guess_hash[:4] == "0000"
```

其中，调用方式也需要相应修改，参考如下。

```
@app.route('/mine', methods=['GET'])
def mine():
    # 获取最后一个区块
    last_block = blockchain.last_block
    # 使用修正后的 proof_of_work 方法
    proof = blockchain.proof_of_work(last_block) # 传入整个区块，而不是仅 last_proof
    # 其余代码保持不变...
```

3. 本次实验为模拟实验，请指出一点未提到的与实际情况不同的地方。

本次区块链模拟实验在一个理想的网络环境中进行，其最核心的简化在于完全忽略了真实分布式网络中的延迟与异步性问题。在实际的区块链网络（如比特币）中，全球节点间的通信存在显著且不确定的网络延迟，这会导致新区块不能同时到达所有节点，从而经常产生临时分叉，并使共识的达成是一个“概率性最终”而非即时确定的过程。本实验通过直接的 HTTP 调用实现即时同步，无法体现这种由网络延迟自然引发的链分叉与竞争状态。此外，实验中使用简单的客户端-服务器通信模式，也极大地简化了真实区块链所依赖的复杂对等网络协议及其面临的网络层挑战。因此，本实验完美模拟了区块链的数据结构与算法逻辑，但未能展现其在不可靠的全球分布式网络中实现共识这一核心挑战。

4. 写出你对区块链的或者理解感受（100 字）。

区块链本质上是一种基于密码学哈希函数、分布式网络与共识算法构建的不可篡改的时序账本技术。其核心创新在于通过去中心化的节点协作与共识机制，在无须可信第三方介入的条件下，建立了具备高度抗篡改性、可追溯性的数据确权与状态验证体系。该技术不仅实现了资产在数字环境中的可靠转移与记录，更通过可编程的智能合约扩展了自动化契约执行的能力，从而为金融、供应链、身份认证等多领域提供了去中介化、可审计的新型基础设施范式，体现了计算逻辑与制度信任的深度融合。

六、实验问题以及解决过程

本实验中的问题主要集中于代码编写方面。实验初期，由于对区块链机制理解尚浅，即便有开发指南作为参考，在代码实现上仍遇到诸多问题。后续通过深入学习 <https://github.com/dvf/blockchain> 上 blockchain.py 的代码，不仅解决了这些具体问题，更对区块链的核心原理有了更深层的认识。

七、实验小结

通过本次实验，我深入理解了区块链技术。我通过查阅资料，对区块链的发展历程及其作为“分布式不可篡改账本”的核心特征形成了初步认识。我依据教程使用 Python Flask 框架

逐步搭建起一个简易区块链，实现了区块生成、工作量证明、交易记录与 **API** 交互等基本功能。随后，我分析了示例代码中工作量证明算法的缺陷，进一步理解了区块链“环环相扣”的防篡改机制。实验使我直观感受到区块链如何借助密码学与共识算法建立去中介信任，同时也认识到模拟环境在网络延迟、异步共识等方面的高度简化。